GameStats

Plataforma multilenguaje para procesamiento y análisis de eventos de videojuegos

Proyecto — Curso de Lenguajes de Programación

1. Descripción general

GameStats es una plataforma para recibir, procesar y analizar los eventos generados durante partidas de videojuegos. El sistema debe ser capaz de trabajar con juegos de naturalezas muy diferentes —por ejemplo, carreras, ajedrez, juegos de cartas, estrategia, deportes o combate— sin que el núcleo de análisis dependa de un único juego.

La idea central del proyecto es separar la generación de eventos del análisis de dichos eventos. Se asume que, en un escenario real, otro sistema o componente externo podría administrar los juegos y producir los eventos. GameStats recibe esos eventos mediante un formato definido por el equipo y debe convertirlos en información útil sobre las partidas, los jugadores y su desempeño.

Cada grupo deberá diseñar su propio conjunto de eventos y estadísticas, de manera que su propuesta tenga sentido para uno o varios tipos de juegos. No se pretende que todos los grupos implementen los mismos juegos ni las mismas estadísticas. Lo que sí debe ser común a todos los grupos es la arquitectura y distribución de responsabilidades entre los lenguajes Go y Scala establecida en este enunciado.

2. Problema a resolver

Un sistema de videojuegos genera continuamente información sobre lo que sucede durante sus partidas. Una partida puede producir cientos o miles de eventos: jugadores que se conectan, acciones, movimientos, ataques, anotaciones, adquisición de objetos, cambios de posición, cumplimiento de objetivos, desconexiones y finalización de la partida, entre muchos otros.

El reto consiste en construir GameStats como una plataforma capaz de recibir este flujo de información y producir análisis relevantes. El sistema no debe limitarse a contar eventos. Debe reconstruir información de las partidas cuando sea necesario, transformar colecciones de eventos, agrupar información, calcular métricas y detectar situaciones o patrones definidos por el equipo.

3. Concepto fundamental: eventos

Un evento representa un hecho ocurrido dentro de una partida. Cada evento debe contener información común que permita identificarlo y relacionarlo con un juego, una partida y, cuando corresponda, un jugador. Además, puede contener información específica de la acción realizada.

Como referencia, un evento podría contener campos como:

- eventId: identificador único del evento.
- timestamp: instante en que ocurrió.
- gameId y gameVersion: juego que produjo el evento.
- matchId: partida a la que pertenece.
- playerId: jugador asociado, cuando corresponda.
- type: tipo general del evento.
- action: acción específica, cuando corresponda.
- data: información específica del juego o de la acción.

La estructura exacta y los tipos de eventos serán definidos por cada grupo en función del análisis de juegos hecho y de la propuesta de formato establecida. Se espera que exista una estructura común suficiente para que GameStats pueda procesar eventos provenientes de diferentes partidas y, potencialmente, de diferentes juegos.

4. Ejemplos de relaciones entre eventos y juegos

Los siguientes ejemplos son ilustrativos y no constituyen una lista obligatoria. El propósito es mostrar que un mismo concepto de evento puede manifestarse de diferentes formas según el tipo de juego.

Tipo de juego	Evento posible	Información asociada	Estadísticas posibles
Carreras	LAP_COMPLETED	vuelta, tiempo, posición, velocidad	mejor vuelta, promedio, evolución de posición
Ajedrez	MOVE	origen, destino, notación, tiempo	tiempo por movimiento, capturas, duración
Combate	PLAYER_ELIMINATED	objetivo, arma, daño, distancia	eliminaciones, K/D, daño promedio
Fútbol/deportes	PASS / GOAL	jugador destino, distancia, éxito	efectividad, asistencias, goles
Cartas	CARD_PLAYED	carta, tipo, costo, resultado	cartas usadas, efectividad, recursos
Estrategia	OBJECTIVE_COMPLETED	objetivo, recursos, duración	objetivos completados, eficiencia, recursos

También pueden existir eventos generales, independientes de la naturaleza del juego, como MATCH_STARTED, MATCH_FINISHED, PLAYER_JOINED, PLAYER_LEFT, PLAYER_DISCONNECTED o PLAYER_RECONNECTED. Cada grupo deberá determinar cuáles son útiles para su propuesta.

5. Diseño libre de eventos y estadísticas

Una parte importante del proyecto es el diseño realizado por cada equipo. No se entregará un catálogo cerrado de eventos ni una lista predeterminada de estadísticas. Cada grupo deberá seleccionar uno o varios tipos de videojuegos y justificar qué información necesita capturar para realizar análisis significativos.

- Definir al menos un modelo de eventos coherente y documentado.
- Definir las relaciones entre eventos, partidas y jugadores.
- Definir estadísticas o métricas que realmente puedan derivarse de los eventos.
- Definir al menos una regla o análisis que requiera considerar una secuencia o combinación de eventos, y no únicamente un evento aislado.
- Generar datos de prueba suficientes para demostrar el funcionamiento del sistema.

Por ejemplo, en un juego de carreras no sería suficiente almacenar solamente la posición final. Podrían registrarse vueltas completadas, tiempos, cambios de posición y penalizaciones para posteriormente calcular consistencia, mejor vuelta o evolución durante la carrera. De manera similar, en un juego de estrategia podrían relacionarse acciones, recursos y objetivos para medir eficiencia.

6. Arquitectura y tecnologías obligatorias

Todos los grupos deberán implementar la misma distribución general de responsabilidades. La elección de los juegos, eventos, métricas y reglas es libre, pero no lo es la asignación principal de responsabilidades entre los lenguajes.

6.1 Módulo de ingestión y procesamiento — Go

El módulo desarrollado en **Go** será responsable de recibir los eventos generados por una fuente externa, organizarlos y prepararlos para el análisis posterior.

- **Recibir o cargar los eventos producidos por una fuente externa.**

El grupo deberá definir un **formato de entrada de eventos**, por ejemplo JSON. Los eventos podrían llegar desde archivos, una API REST, un flujo de mensajes u otro mecanismo. **No es necesario implementar el videojuego que genera los eventos**; se deberá crear un simulador o conjunto de datos que represente dicha fuente externa.

- **Validar la estructura básica de los eventos.**

Go deberá comprobar que los eventos recibidos contienen la información mínima necesaria, por ejemplo: identificador del evento, fecha/hora, juego, partida, jugador y tipo de evento. Los eventos inválidos deberán ser identificados y tratados adecuadamente, por ejemplo, registrándolos o rechazándolos.

- **Administrar el flujo de procesamiento.**

Go deberá controlar el recorrido que siguen los eventos desde que son recibidos hasta que son enviados al módulo Scala. Esto puede incluir recibir eventos, clasificarlos por partida o juego, almacenarlos temporalmente, agruparlos y decidir cuándo están listos para ser analizados.

No significa que Go deba realizar las estadísticas que corresponden a Scala.

- **Mantener el estado necesario para reconstruir o agrupar información de partidas.**

Go deberá mantener temporalmente información que permita relacionar eventos que pertenecen a una misma partida. Por ejemplo, si recibe MATCH_STARTED, varios PLAYER_ACTION y finalmente MATCH_FINISHED, deberá ser posible identificar que todos esos eventos forman parte de una misma partida. También puede utilizar este estado para mantener información como jugadores activos, eventos acumulados o estado básico de una partida.

- **Manejar procesamiento concurrente para varios juegos o partidas que se ejecuten en paralelo.**

La solución deberá demostrar el uso de **concurrencia en Go**. Por ejemplo, GameStats podría recibir simultáneamente eventos correspondientes a varias partidas o juegos diferentes. El diseño deberá permitir procesar estas entradas concurrentemente utilizando mecanismos propios de Go, como **goroutines y channels**, cuando sean apropiados.

- **Preparar y enviar los datos que serán analizados por Scala.**

Una vez organizados los eventos, Go deberá entregarlos al módulo Scala mediante un mecanismo de comunicación definido por el grupo. Por ejemplo, puede enviar eventos mediante una API, archivos estructurados o mensajería. Go debe encargarse de la **ingestión y preparación**, mientras que Scala será responsable del **análisis funcional**.

6.2 Módulo de análisis — Scala

El módulo desarrollado en **Scala** será responsable de transformar los eventos recibidos en información estadística y analítica sobre las partidas.

- **Recibir los eventos o datos preparados por el módulo Go.**

  Scala deberá recibir la información proveniente de Go y convertirla en las estructuras de datos que utilizará para realizar el análisis.

- **Realizar transformaciones utilizando colecciones y operaciones funcionales.**

  El procesamiento deberá aprovechar las características del paradigma funcional de Scala. Por ejemplo, transformar una colección de eventos en otra colección que contenga únicamente determinados tipos de eventos, información agrupada por jugador o información resumida por partida.

- **Aplicar filtrado, mapeo, agrupación, reducción y composición de funciones cuando sean apropiados.**

  Las estadísticas deberán implementarse utilizando, cuando corresponda, operaciones como filter, map, flatMap, groupBy, fold, reduce, collect y composición de funciones. **No se trata de utilizar todas obligatoriamente**, sino de utilizarlas de manera justificada para resolver el problema.

- **Calcular las estadísticas definidas por el equipo.**

Cada grupo deberá definir las métricas apropiadas para los juegos que haya seleccionado. Por ejemplo, un juego de carreras podría calcular tiempo promedio por vuelta, mientras que un juego de combate podría calcular eliminaciones, daño promedio o relación entre eliminaciones y derrotas.

- **Implementar reglas o detección de patrones de la solución.**
  El sistema deberá identificar situaciones que requieran analizar **varios eventos relacionados**, no solamente contar eventos individuales. Por ejemplo, detectar una remontada cuando un jugador estuvo en desventaja y posteriormente ganó, identificar una determinada secuencia de acciones o detectar un comportamiento definido por el grupo.
- **Generar resultados estructurados para ser consultados o almacenados.**
  Los resultados del análisis deberán producirse en un formato estructurado, por ejemplo JSON, de manera que puedan ser posteriormente consultados por otro componente, almacenados o mostrados en una interfaz. Los resultados podrían incluir estadísticas por jugador, por partida, por equipo, por juego o análisis específicos definidos por el grupo.

7. Comunicación entre módulos

Los módulos Go y Scala deberán ejecutarse como componentes independientes y comunicarse mediante un mecanismo definido por el equipo, por ejemplo HTTP/REST, intercambio de archivos estructurados o mensajería. La alternativa elegida deberá estar documentada.

El formato de intercambio deberá ser suficientemente claro para que un componente externo pueda producir eventos sin conocer la implementación interna de GameStats.

8. Fuente de datos y simulación

No es necesario implementar los videojuegos. GameStats deberá contar con una fuente de eventos que simule lo que produciría un sistema de juegos externo. Esta fuente deberá generar datos realistas y variados.

- Varias partidas.
- Varios jugadores.
- Diferentes resultados y situaciones.
- Eventos en diferentes momentos de una partida.
- Partidas de diferentes juegos en paralelo
- Casos normales y casos que permitan comprobar reglas o situaciones especiales.
- Datos suficientes para demostrar que las estadísticas son significativas.

9. Funcionalidades mínimas

1. Registrar o recibir eventos de partidas.
2. Validar y procesar eventos.
3. Identificar juegos, partidas y jugadores.
4. Procesar múltiples eventos pertenecientes a una misma partida.
5. Calcular un conjunto de estadísticas definidas por el grupo.
6. Implementar al menos una estadística agregada por jugador.
7. Implementar al menos una estadística agregada por partida.
8. Implementar al menos una estadística que requiera combinar o relacionar diferentes tipos de eventos.
9. Implementar al menos una regla o detección de patrón basada en una secuencia o combinación de eventos.
10. Permitir consultar o presentar los resultados generados por el sistema.

10. Desarrollo, versionamiento y deployment

El proyecto deberá demostrar un flujo básico de desarrollo y entrega de software, no solamente la ejecución local del programa.

- Repositorio Git con historial de desarrollo.
- Construcción reproducible de los módulos.
- Contenerización mediante Docker.
- Configuración para ejecutar los componentes de GameStats de manera integrada.
- Pipeline de CI/CD que ejecute al menos compilación y administración de contenedores, archivos, carpetas, etc.
- Deployment de una versión funcional del sistema en un entorno definido por el equipo.

11. Entregables

- Código fuente completo de los módulos Go y Scala.
- Especificación del formato de eventos diseñado por el equipo.
- Descripción de los tipos de juegos considerados y justificación de los eventos seleccionados.
- Descripción de las estadísticas y reglas implementadas.
- Dockerfiles y configuración necesaria para ejecutar el sistema.
- Configuración del pipeline CI/CD.
- Documentación de instalación, ejecución y deployment.
- Demostración funcional del sistema.

12. Consideraciones y restricciones

- La implementación debe utilizar Go y Scala como lenguajes principales.
- Go debe concentrar la ingestión, control del flujo y procesamiento imperativo.
- Scala debe concentrar el procesamiento analítico y funcional.
- Los videojuegos no forman parte del alcance; se simulará la fuente de eventos.
- El diseño de los juegos, eventos, estadísticas y reglas es responsabilidad de cada grupo.
- La solución debe ser suficientemente genérica para que un nuevo conjunto de eventos pueda incorporarse sin modificar completamente el sistema.
- Las decisiones de arquitectura y formato deberán estar justificadas técnicamente.

13. Objetivo académico

El proyecto busca que el estudiante experimente con dos paradigmas de programación en un problema integrado. La implementación deberá evidenciar cómo una misma problemática puede abordarse de manera diferente según el paradigma y el lenguaje utilizado.

Además de la implementación, se espera que el equipo pueda justificar qué responsabilidades son apropiadas para un enfoque imperativo y cuáles se benefician de un enfoque funcional, así como evaluar las ventajas y limitaciones de ambas aproximaciones dentro de un sistema real compuesto por varios servicios.

14. Aspectos Administrativos

- El proyecto puede realizarse en grupos de máximo dos personas. Bajo ninguna circunstancia se permitirán ***COPIAS*** de trabajos.
- Documentación: la solicitada en el documento. Formato libre.
- Fecha de entrega: domingo 18 de OCTUBRE de 2026.
- Hora de entrega: antes de las 10:00 pm a través por el Tec Digital.